

Assignment 4: Implementation of a Basic RAG System

Practical Module

Retrieval-Augmented Generation (RAG) — Practical Implementation

Course

B.Tech – Computer Engineering

Faculty

Dr. A. V. Brahmane

College

Sanjivani College of Engineering

1. Title of the Experiment

Implementation of a Basic Retrieval-Augmented Generation (RAG) System using Python, LangChain and FAISS

2. Aim

To design and implement a basic **Retrieval-Augmented Generation (RAG)** system that retrieves relevant information from a document and uses a Large Language Model (LLM) to generate an accurate answer based on the retrieved information.

3. Problem Statement

Large Language Models can generate very good answers, but they may not know the information contained in a student's private or newly created documents.

In this experiment, we will create a RAG system that can:

1. Read a document.
2. Divide the document into smaller chunks.
3. Convert the chunks into numerical vectors called embeddings.
4. Store the vectors in a FAISS vector database.
5. Search for the most relevant chunks when the user asks a question.
6. Send the retrieved information to an LLM.
7. Generate an answer based on the retrieved document information.

The overall idea is:

Document → Chunking → Embedding → FAISS → Retrieval → LLM → Answer

The assignment describes RAG as the combination of **Retrieval + Generation**, where retrieved document information is supplied to the LLM before generating the final answer.

4. Learning Objectives

After completing this experiment, students will be able to:

- Understand the basic concept of RAG.
- Understand the architecture of a RAG system.
- Load text/PDF documents using LangChain.
- Split large documents into smaller chunks.
- Generate text embeddings.
- Create a FAISS vector store.
- Perform similarity-based retrieval.

- Connect the retriever with an LLM.
- Create a custom prompt for RAG.
- Generate answers using retrieved information.
- Display source documents used for an answer.
- Experiment with different values of k.
- Identify common RAG implementation problems.

5. Software and Hardware Requirements

Software Requirements

- Python 3.x
- VS Code / Jupyter Notebook / Google Colab
- Internet connection
- OpenAI API key if using OpenAI LLM
- Required Python libraries

Main Libraries

Library	Purpose
LangChain	Connects different RAG components
sentence-transformers	Generates embeddings
FAISS	Stores and searches vectors
OpenAI	Provides LLM access
PyPDF2 / PyPDF	Reads PDF files
NumPy	Numerical operations
python-dotenv	Reading API keys from .env file

The assignment specifically identifies LangChain, sentence-transformers, FAISS, OpenAI/Hugging Face and PDF-processing libraries as the main components.

6. What is RAG?

6.1 Simple Definition

RAG stands for Retrieval-Augmented Generation.

It combines two major operations:

Retrieval

Finding the most relevant information from a document collection.

Generation

Using an LLM to generate a natural-language answer from the retrieved information.

Therefore:

RAG = Retrieval + Generation

7. Why Do We Need RAG?

Suppose we have a PDF containing college information.

A student asks:

"What is the attendance requirement mentioned in the document?"

A normal LLM may not know the answer because the information is present only in our PDF.

RAG solves this problem.

The RAG system:

1. Reads the PDF.
2. Finds the part containing attendance information.
3. Sends that information to the LLM.
4. The LLM generates the answer.

Therefore, the LLM does not have to depend only on its previously learned knowledge.

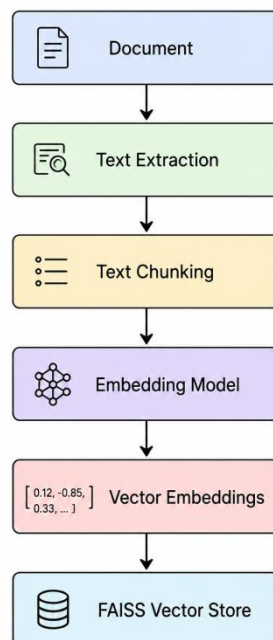
The assignment explains this using an **open-book examination analogy**: the LLM behaves like a student who can refer to the textbook before answering.

8. Basic RAG Architecture

The RAG system has two major phases.

Phase 1: Offline Indexing

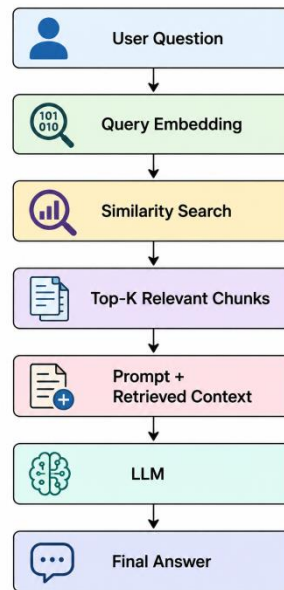
This phase is performed when documents are added.



This phase normally does not need to be repeated for every question.

Phase 2: Online Querying

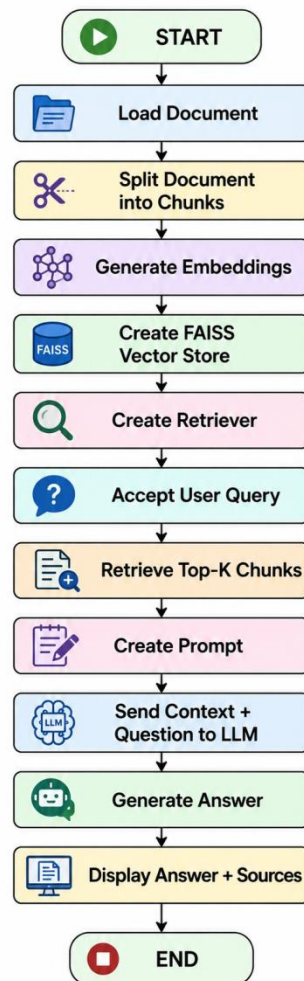
This phase occurs whenever the user asks a question.



The assignment identifies the same distinction between **offline indexing** and **online querying**.

9. Complete RAG Workflow

The complete implementation consists of the following steps:



The assignment represents this sequence as:

LOAD → SPLIT → EMBED → STORE → QUERY → RETRIEVE → GENERATE → OUTPUT.

10. RAG vs Fine-Tuning

Students often confuse RAG with fine-tuning.

Feature	RAG	Fine-Tuning
Knowledge update	Add documents	Retrain model
Training required	No	Yes
GPU requirement	Low	Usually high
Source information	Can be shown	Usually not directly available
Data required	Documents	Training examples
Best for	Changing knowledge	Changing model behaviour/style

For this assignment, RAG is preferred because the knowledge can be updated simply by changing the document collection.

11. Step 1 — Environment Setup

11.1 Create a Project Folder

Create the following folder structure:

RAG_Assignment/

```
|
|— data/
|   |— ml_notes.txt
|— rag_system.py
|— faiss_index/
```

The data folder contains the document that will be used by the RAG system.

12. Install Required Libraries

Open the terminal in VS Code or a Google Colab cell and install the required packages.

pip install langchain sentence-transformers faiss-cpu openai pypdf python-dotenv

The original assignment uses the same basic dependency set, including LangChain, sentence-transformers, FAISS, OpenAI and PDF support.

13. API Key Setup

If OpenAI is used as the LLM, an API key is required.

Important Rule

Never publish or share your API key.

Do not write your actual API key in a public GitHub repository.

A .env file can be used.

Example:

OPENAI_API_KEY=your_api_key_here

Then Python can read the key using:

import os

from dotenv import load_dotenv

load_dotenv()

api_key = os.getenv("OPENAI_API_KEY")

The assignment also recommends using environment variables instead of directly exposing API keys in source code.

14. Step 2 — Document Loading

Objective

The first practical task is to load our document into Python.

LangChain represents loaded content using a **Document object**.

A Document generally contains:

page_content

metadata

page_content

Contains the actual text.

metadata

Contains additional information such as:

- filename
- page number
- source

The assignment explains that different loaders can be used for TXT, PDF and DOCX files.

14.1 Loading a TXT File

Example:

```
from langchain.document_loaders import TextLoader
```

```
loader = TextLoader("data/ml_notes.txt")
```

```
documents = loader.load()
```

```
print("Number of documents:", len(documents))
```

```
print(documents[0].page_content[:500])
```

```
print(documents[0].metadata)
```

Explanation

```
loader = TextLoader("data/ml_notes.txt")
```

Creates a loader object and tells LangChain where the file is located.

```
documents = loader.load()
```

Reads the file.

```
documents[0].page_content
```

Displays the text.

```
documents[0].metadata
```

Displays source information.

15. Loading a PDF

For a PDF document:

```
from langchain.document_loaders import PyPDFLoader
```

```
loader = PyPDFLoader("data/sample.pdf")
```

```
documents = loader.load()
```

```
print("Number of pages:", len(documents))
```

```
print(documents[0].page_content[:500])
```

Normally, each page can be represented as a separate Document object.

16. Step 3 — Text Chunking

Why Do We Need Chunking?

A document may contain hundreds of pages.

We cannot simply send the entire document to the LLM every time.

Therefore, we divide the document into smaller pieces.

These smaller pieces are called **chunks**.

Example:

Large Document

↓

Chunk 1	
Chunk 2	
Chunk 3	
Chunk 4	

The assignment recommends starting with approximately **500 characters** and using an overlap of about **100 characters**.

17. Chunk Size

`chunk_size` specifies the approximate maximum size of each chunk.

Example:

`chunk_size=500`

means we try to create chunks of approximately 500 characters.

Smaller chunks

Advantages:

- More precise retrieval
- Less unnecessary information

Disadvantages:

- Context may be lost

Larger chunks

Advantages:

- More context
- Related information stays together

Disadvantages:

- Retrieval may become less precise
 - More information is sent to the LLM
-

18. Chunk Overlap

Consider:

Chunk 1:

Machine learning is a method...



Chunk 2:

...method used to learn patterns...

Some information is repeated between chunks.

This repeated area is called **overlap**.

Example:

`chunk_overlap=100`

Overlap helps prevent important information from being lost at the boundary of two chunks.

19. Implement Text Chunking

`from langchain.text_splitter import RecursiveCharacterTextSplitter`

```
splitter = RecursiveCharacterTextSplitter(  
    chunk_size=500,  
    chunk_overlap=100  
)
```

```
chunks = splitter.split_documents(documents)
```

```
print("Total chunks:", len(chunks))
```

```
print("\nFirst Chunk:")  
print(chunks[0].page_content)
```

```
print("\nMetadata:")  
print(chunks[0].metadata)
```

20. Why RecursiveCharacterTextSplitter?

The splitter attempts to preserve meaningful text boundaries.

It can split text progressively using separators such as:

Paragraph



Line



Sentence/word boundary

This is better than simply cutting the document at an arbitrary position.

The assignment specifically uses RecursiveCharacterTextSplitter for this purpose.

21. Step 4 — Generate Embeddings

What is an Embedding?

An embedding is a numerical representation of text.

For example:

"Machine learning is a branch of AI"



Embedding Model



[0.021, -0.143, 0.087, ...]

The numerical representation is called a **vector**.

Texts with similar meanings tend to have similar vectors.

22. Why Do We Need Embeddings?

Suppose our document contains:

"A car is a vehicle used for transportation."

The user asks:

"What is an automobile?"

The words car and automobile are different.

Keyword search may have difficulty finding the relationship.

An embedding model tries to understand their semantic similarity.

Therefore:

Document Text



Embedding



Vector

and

User Query



Embedding



Query Vector

The two vectors can then be compared.

23. Embedding Model Used

The assignment uses:

all-MiniLM-L6-v2

This is a lightweight sentence-transformer model.

According to the assignment, it generates **384-dimensional vectors**.

24. Generate Embeddings in Python

```
from langchain.embeddings import HuggingFaceEmbeddings
```

```
embeddings = HuggingFaceEmbeddings(  
    model_name="all-MiniLM-L6-v2"  
)
```

Test the embedding:

```
test_vector = embeddings.embed_query(
    "What is machine learning?"
)
```

```
print("Vector length:", len(test_vector))
```

```
print(test_vector[:5])
```

Expected result:

Vector length: 384

The exact numerical values may be different depending on the model/runtime.

25. Two Important Embedding Functions

embed_documents()

Used to convert multiple document chunks into vectors.

```
vectors = embeddings.embed_documents(
    ["Machine learning is AI", "Deep learning uses neural networks"]
)
```

embed_query()

Used for a single user question.

```
query_vector = embeddings.embed_query(
    "What is machine learning?"
)
```

Both document chunks and user queries must be represented in the same embedding space so that they can be compared.

26. Step 5 — Create FAISS Vector Store

What is FAISS?

FAISS stands for:

Facebook AI Similarity Search

It is a library used for efficient similarity search over vectors.

In our RAG system:

Chunks

↓

Embeddings

↓

FAISS

FAISS stores the vectors and helps us find the vectors most similar to a query vector.

The assignment uses FAISS as the main vector store.

27. Why FAISS?

FAISS is useful because:

- It is open source.
- It supports fast similarity search.

- It can handle a large number of vectors.
- It integrates with LangChain.
- The index can be saved to disk.

28. Create FAISS Vector Store

from langchain.vectorstores import FAISS

```
vectorstore = FAISS.from_documents(  
    chunks,  
    embeddings  
)
```

```
print(  
    "Number of vectors:",  
    vectorstore.index.ntotal  
)
```

This important line:

FAISS.from_documents(chunks, embeddings)

performs the following operations:

Chunks

↓

Embedding Model

↓

Vectors

↓

FAISS Index

29. Save the FAISS Index

Instead of creating the index every time, we can save it.

```
vectorstore.save_local("faiss_index")
```

```
print("FAISS index saved successfully!")
```

The assignment notes that the saved index consists of files such as:

faiss_index/

index.faiss

index.pkl

This allows the index to be loaded later without repeating the complete embedding process.

30. Load an Existing FAISS Index

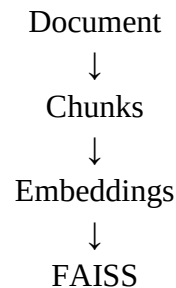
```
vectorstore = FAISS.load_local(  
    "faiss_index",  
    embeddings,  
    allow_dangerous_deserialization=True  
)
```

Then:

```
print("FAISS index loaded successfully!")
```

31. Step 6 — Retrieval

Now we have:

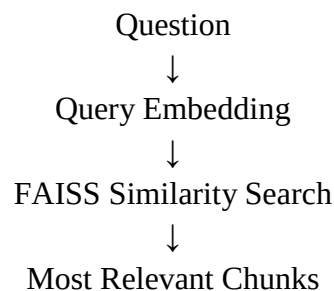


The next task is to retrieve relevant chunks.

Suppose the user asks:

"What is gradient descent?"

The system converts this question into an embedding and searches the FAISS database.



32. What is Top-K Retrieval?

k represents the number of chunks that should be retrieved.

For example:

k=3

means:

Return the three most relevant chunks.

The assignment recommends experimenting with values such as 3–5 and specifically requires comparison of k=1, k=3, and k=5.

33. Create a Retriever

```
retriever = vectorstore.as_retriever(  
    search_kwargs={"k": 3}  
)
```

Here:

k = 3

means three relevant chunks will normally be returned.

34. Test Retrieval

```
query = "What is a neural network?"
```

```
results = retriever.get_relevant_documents(query)
```

```
for i, doc in enumerate(results):  
    print("\n--- Chunk", i + 1, "---")  
    print(doc.page_content)  
    print("Source:", doc.metadata.get("source"))
```

This is an important debugging step.

Before blaming the LLM for a wrong answer, check whether the correct information was actually retrieved.

The assignment explicitly emphasizes that many RAG problems are retrieval problems.

35. Step 7 — Configure the LLM

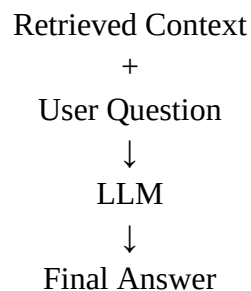
The LLM is responsible for generating the final answer.

Important point:

The LLM itself does not perform document retrieval.

The retriever finds the relevant chunks.

The LLM receives:



The assignment presents OpenAI GPT as one option and Hugging Face local models as another option.

36. Temperature

Temperature controls how varied the generated output can be.

For factual question-answering, the assignment recommends:

```
temperature=0
```

This makes the response more deterministic.

For creative tasks, a higher value may be used.

For our RAG experiment:

```
temperature = 0
```

is appropriate.

37. OpenAI LLM Example

The assignment uses:

```
from langchain.chat_models import ChatOpenAI
```

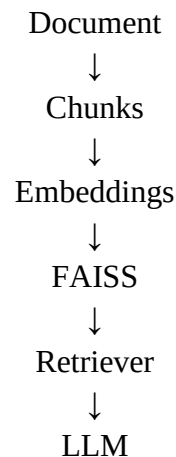
```
llm = ChatOpenAI(  
    model_name="gpt-3.5-turbo",  
    temperature=0,
```

```
    max_tokens=512
)
```

The exact availability of a particular OpenAI model/API can change over time, so students should use the model currently available to their API account if the specified model is unavailable.

38. Step 8 — Build the RAG Chain

Now we have all the important components:



We need to connect them.

The assignment uses LangChain's RetrievalQA approach.

Example:

```
from langchain.chains import RetrievalQA
```

```
qa_chain = RetrievalQA.from_chain_type(
    llm=llm,
    chain_type="stuff",
    retriever=retriever,
    return_source_documents=True
)
```

39. What Does chain_type="stuff" Mean?

Suppose the retriever returns:

Chunk 1

Chunk 2

Chunk 3

The stuff chain puts these chunks together into one prompt.

Conceptually:

Context:

Chunk 1

Chunk 2

Chunk 3

Question:

What is gradient descent?

Generate Answer

For a small number of retrieved chunks, this approach is simple and suitable for this assignment.

40. Ask a Question

```
query = "What is gradient descent?"
```

```
result = qa_chain({
    "query": query
})
```

```
print("QUESTION:")
print(query)
```

```
print("\nANSWER:")
print(result["result"])
```

41. Display Source Documents

One important advantage of RAG is that we can see the documents used to generate the answer.

```
print("\nSOURCE DOCUMENTS:")
```

```
for i, doc in enumerate(
    result["source_documents"]
):
```

```
    print("\nSource", i + 1)
    print(doc.metadata)
```

```
    print(doc.page_content[:300])
```

This helps us verify whether the answer is actually supported by the retrieved document. The assignment specifically requires source references in the sample output.

42. Step 9 — Custom Prompt Engineering

A default prompt may not be sufficient.

We can create a custom prompt that tells the LLM:

- Use only retrieved information.
- Do not invent information.
- If the answer is not present, say that it is unknown.

Example:

```
from langchain.prompts import PromptTemplate
```

```
template = ""
```

You are a helpful academic assistant.

Use ONLY the information given in the context
to answer the question.

If the answer is not available in the context,
say:

"I don't know based on the provided documents."

Do not use outside knowledge.

Context:
{context}

Question:
{question}

Answer:
""

```
prompt = PromptTemplate(  
    template=template,  
    input_variables=["context", "question"]  
)
```

The original assignment recommends this type of prompt restriction to reduce hallucination.

43. Why Is the Custom Prompt Important?

Without a restriction, an LLM may answer from its general knowledge.

For example, suppose our document does not contain:

"Who invented a particular algorithm?"

The LLM might still try to answer.

With our custom prompt:

Answer only from the provided context.

the expected response becomes:

I don't know based on the provided documents.

This makes the system more grounded.

44. Complete RAG Program

The following program combines the major steps discussed above.

```
# =====  
# BASIC RAG SYSTEM  
# Assignment 4 - Agentic AI & LLM Systems
```

```

# =====

import os

# -----
# 1. API KEY
# -----

os.environ["OPENAI_API_KEY"] = "YOUR_API_KEY"

# -----
# 2. IMPORT LIBRARIES
# -----

from langchain.document_loaders import TextLoader
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain.embeddings import HuggingFaceEmbeddings
from langchain.vectorstores import FAISS
from langchain.chains import RetrievalQA
from langchain.chat_models import ChatOpenAI

# -----
# 3. LOAD DOCUMENT
# -----

loader = TextLoader("data/ml_notes.txt")

documents = loader.load()

print("Documents loaded:", len(documents))

# -----
# 4. SPLIT DOCUMENT INTO CHUNKS
# -----

splitter = RecursiveCharacterTextSplitter(
    chunk_size=500,
    chunk_overlap=100
)

chunks = splitter.split_documents(documents)

print("Total chunks:", len(chunks))

```

```
# -----
# 5. CREATE EMBEDDING MODEL
# -----

embeddings = HuggingFaceEmbeddings(
    model_name="all-MiniLM-L6-v2"
)

print("Embedding model loaded.")

# -----
# 6. CREATE FAISS VECTOR STORE
# -----

vectorstore = FAISS.from_documents(
    chunks,
    embeddings
)

print(
    "Vectors stored:",
    vectorstore.index.ntotal
)

# -----
# 7. SAVE VECTOR STORE
# -----

vectorstore.save_local("faiss_index")

print("FAISS index saved.")

# -----
# 8. CREATE RETRIEVER
# -----

retriever = vectorstore.as_retriever(
    search_kwargs={"k": 3}
)

# -----
# 9. CREATE LLM
# -----
```

```

llm = ChatOpenAI(
    model_name="gpt-3.5-turbo",
    temperature=0,
    max_tokens=512
)

print("LLM initialized.")

# -----
# 10. CREATE RAG CHAIN
# -----

qa_chain = RetrievalQA.from_chain_type(
    llm=llm,
    chain_type="stuff",
    retriever=retriever,
    return_source_documents=True
)

# -----
# 11. ASK QUESTION
# -----

query = "What is gradient descent?"

result = qa_chain({
    "query": query
})

# -----
# 12. DISPLAY ANSWER
# -----

print("\n=====")
print("QUESTION")
print("=====")

print(query)

print("\n=====")
print("ANSWER")
print("=====")

print(result["result"])

```

```

# -----
# 13. DISPLAY SOURCES
# -----

print("\n=====")
print("SOURCE DOCUMENTS")
print("=====")

for i, doc in enumerate(
    result["source_documents"]
):

    print("\nSource", i + 1)
    print("Metadata:", doc.metadata)
    print("Content:")
    print(doc.page_content[:300])

```

The structure above follows the complete implementation sequence presented in the assignment: loading, chunking, embedding, FAISS storage, retrieval, LLM setup and RAG-chain execution.

45. Understanding the Complete Code

The complete program can be understood as eight blocks.

Block 1 — Import Libraries

```

from langchain.document_loaders import TextLoader
Loads the document.

from langchain.text_splitter import RecursiveCharacterTextSplitter
Splits the document.

from langchain.embeddings import HuggingFaceEmbeddings
Creates embeddings.

from langchain.vectorstores import FAISS
Creates the vector database.

from langchain.chains import RetrievalQA
Connects retrieval and generation.

from langchain.chat_models import ChatOpenAI
Connects the LLM.

```

46. Block 2 — Load Document

```

loader = TextLoader("data/ml_notes.txt")
documents = loader.load()

```

The document is converted into LangChain Document objects.

47. Block 3 — Chunking

```

splitter = RecursiveCharacterTextSplitter(

```

```
    chunk_size=500,  
    chunk_overlap=100  
)
```

```
chunks = splitter.split_documents(documents)
```

The large document is converted into manageable pieces.

48. Block 4 — Embedding + FAISS

```
embeddings = HuggingFaceEmbeddings(  
    model_name="all-MiniLM-L6-v2"  
)
```

```
vectorstore = FAISS.from_documents(  
    chunks,  
    embeddings  
)
```

This converts chunks into vectors and stores them in FAISS.

49. Block 5 — Retriever

```
retriever = vectorstore.as_retriever(  
    search_kwargs={"k": 3}  
)
```

This tells the system:

Retrieve the top 3 relevant chunks.

50. Block 6 — LLM

```
llm = ChatOpenAI(  
    model_name="gpt-3.5-turbo",  
    temperature=0  
)
```

The LLM is responsible for generating the final answer.

51. Block 7 — RAG Chain

```
qa_chain = RetrievalQA.from_chain_type(  
    llm=llm,  
    chain_type="stuff",  
    retriever=retriever,  
    return_source_documents=True  
)
```

This connects:

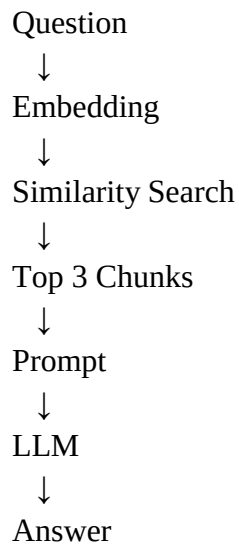
Retriever + LLM

into one pipeline.

52. Block 8 — Query

```
result = qa_chain({
    "query": "What is gradient descent?"
})
```

Internally, the system performs approximately:



53. Understanding Similarity Search

The embedding model represents text as vectors.

Suppose:

Query Vector = Q

Document Vector 1 = D1

Document Vector 2 = D2

Document Vector 3 = D3

The system calculates how similar these vectors are.

The most similar vectors represent the most relevant chunks.

54. Cosine Similarity

Cosine similarity measures the angle between two vectors.

The formula is:

$$A \cdot B$$

Cosine = -----

$$|A| \times |B|$$

A value close to:

1

means high similarity.

A lower value indicates less similarity.

The assignment explains cosine similarity and L2 distance as the major similarity concepts used in vector search.

55. L2 Distance

L2 distance represents the straight-line distance between vectors.

Important rule:

Lower L2 distance

=

More similar

The assignment notes that LangChain's basic FAISS configuration uses L2 distance by default.

56. Experiment — Effect of K

One of the required experiments is to compare:

k = 1

k = 3

k = 5

Experiment 1: k = 1

```
retriever = vectorstore.as_retriever(  
    search_kwargs={"k": 1}  
)
```

Only one chunk is retrieved.

Expected observation

The answer may be highly focused but may miss additional context.

Experiment 2: k = 3

```
retriever = vectorstore.as_retriever(  
    search_kwargs={"k": 3}  
)
```

Three chunks are retrieved.

Expected observation

The LLM receives more context and can generally provide a more complete answer.

Experiment 3: k = 5

```
retriever = vectorstore.as_retriever(  
    search_kwargs={"k": 5}  
)
```

Five chunks are retrieved.

Expected observation

More information is available, but irrelevant or repetitive chunks may also be included.

The assignment specifically requires this comparison as part of the experiment.

57. Experiment Table

Students should prepare a table similar to the following:

Query	k	Retrieved Chunks	Answer Quality	Observation
-------	---	------------------	----------------	-------------

Query 1	1	1	_____	_____
---------	---	---	-------	-------

Query 1	3	3	_____	_____
---------	---	---	-------	-------

Query k Retrieved Chunks Answer Quality Observation

Query 1	5	5	_____	_____
Query 2	1	1	_____	_____
Query 2	3	3	_____	_____
Query 2	5	5	_____	_____
Query 3	1	1	_____	_____
Query 3	3	3	_____	_____
Query 3	5	5	_____	_____

Students should fill this table after actually executing the program.

58. Suggested Test Questions

Use questions whose answers are present in your document.

For example:

1. What is machine learning?

2. What is overfitting?

3. What is gradient descent?

4. What is a neural network?

5. What is backpropagation?

The assignment itself provides similar test questions for evaluating the RAG system.

59. Sample Output Format

Students should capture output in the following format:

=====

QUESTION

=====

What is gradient descent?

=====

ANSWER

=====

Gradient descent is an optimization technique used to minimize the error of a machine learning model by updating model parameters step by step.

=====

SOURCE DOCUMENTS

=====

Source 1:
data/ml_notes.txt

Source 2:
data/ml_notes.txt

Source 3:
data/ml_notes.txt

The actual answer must come from the student's document and therefore may be different.

60. RAG Quality Evaluation

A RAG system should not be evaluated only by asking:

"Does the answer look good?"

We should check three important aspects.

60.1 Faithfulness

Does the answer contain information supported by the retrieved context?

60.2 Answer Relevance

Does the answer actually answer the user's question?

60.3 Context Recall

Did the retriever find the correct information from the document?

The assignment identifies these three aspects as important RAG evaluation criteria.

61. Important Debugging Rule

If the answer is wrong, do not immediately assume that the LLM is the problem.

First print the retrieved chunks.

```
results = retriever.get_relevant_documents(query)
```

```
for doc in results:
```

```
    print(doc.page_content)
```

Ask:

Did the retriever find the correct information?

If the correct information was not retrieved, the LLM cannot reliably answer from that information.

The assignment summarizes this as a practical debugging rule: check retrieval first.

62. Common Errors and Solutions

Error 1 — No module named faiss

Problem

FAISS is not installed.

Solution

pip install faiss-cpu

Error 2 — OpenAI Authentication Error

Problem

API key is missing or incorrect.

Solution

Check:

`os.environ["OPENAI_API_KEY"] = "YOUR_API_KEY"`

or correctly configure the .env file.

Error 3 — Wrong Answer

Possible causes

- Wrong chunks retrieved.
- Chunk size too large.
- Chunk size too small.
- k value too low.
- Document does not contain the answer.
- Prompt does not restrict the LLM.

Possible solutions

Try:

`chunk_size: 500 → 300`

or:

`k: 3 → 5`

The assignment recommends these approaches for poor retrieval quality.

63. Error — Empty Chunks

If you receive an error such as:

`IndexError: list index out of range`

check:

1. Is the file path correct?
2. Is the file empty?
3. Was the file loaded successfully?
4. Is the file encoding correct?

For debugging:

`print(len(documents))`

and:

`print(len(chunks))`

64. Slow First Embedding Run

When the embedding model runs for the first time, it may download the model.

Therefore, the first execution may take longer.

After downloading, the model is normally cached.

The assignment specifically notes this behaviour for all-MiniLM-L6-v2.

65. Hallucination Problem

If the LLM generates information that is not present in the document, we call this a hallucination.

To reduce this problem:

Use a strong prompt

Use ONLY the provided context.

If the answer is not present,

say that you don't know.

Use low temperature

temperature=0

The assignment recommends both approaches.

66. FAISS vs ChromaDB

FAISS is not the only vector store.

Another option is **ChromaDB**.

Feature	FAISS	ChromaDB
Vector search	Yes	Yes
Open source	Yes	Yes
Metadata filtering	Limited/basic	Stronger
Persistent storage	Yes	Yes
Simple similarity search	Excellent	Excellent
Suitable for RAG	Yes	Yes

The assignment describes FAISS as a strong choice for pure similarity search and ChromaDB as useful when metadata filtering and persistent database features are important.

67. Optional ChromaDB Example

Install:

```
pip install chromadb
```

Example:

```
from langchain.vectorstores import Chroma
```

```
vectorstore = Chroma.from_documents(  
    chunks,  
    embeddings,  
    persist_directory="chroma_db"  
)
```

```
vectorstore.persist()
```

This can be used as an alternative to FAISS.

However, **FAISS should be used for the main implementation of this assignment**, because it is the vector store specified in the main pipeline.

68. Practical Implementation Flow

Students should remember the following simple sequence:

STEP 1

Load Document



STEP 2

Split Document



STEP 3

Generate Embeddings



STEP 4

Store Embeddings in FAISS



STEP 5

Create Retriever



STEP 6

Ask Question



STEP 7

Retrieve Relevant Chunks



STEP 8

Send Context + Question to LLM



STEP 9

Generate Answer



STEP 10

Display Sources

69. What Happens Internally?

Suppose the user asks:

"What is overfitting?"

The system performs:

Step 1

Convert question into an embedding.

"What is overfitting?"



Query Vector

Step 2

Compare the query vector with vectors stored in FAISS.

Step 3

Find the most similar chunks.

Top 3 Chunks

Step 4

Create a prompt:

Context:

[Retrieved Chunk 1]

[Retrieved Chunk 2]

[Retrieved Chunk 3]

Question:

What is overfitting?

Step 5

Send the prompt to the LLM.

Step 6

LLM generates the final answer.

70. Advantages of RAG

RAG provides several important advantages:

1. It can use external documents.
2. Knowledge can be updated without retraining the LLM.
3. It can reduce hallucination.
4. It can provide source information.
5. It is suitable for private document-based question answering.
6. It separates knowledge retrieval from language generation.

The assignment emphasizes that RAG keeps the LLM's knowledge mechanism separate from the external document store.

71. Limitations of Basic RAG

A basic RAG system can still produce poor answers if:

- The document is poorly formatted.
- Chunking is inappropriate.
- The wrong chunks are retrieved.
- Too few chunks are retrieved.
- Too many irrelevant chunks are retrieved.
- The prompt is poorly designed.
- The document does not contain the required information.

Therefore:

Good Documents

+

Good Chunking

+

Good Embeddings
+
Good Retrieval
+
Good Prompt
+
Good LLM
=
Better RAG System

72. Assignment Deliverables

Students are required to submit the following.

Deliverable 1 — Working Code

Submit the complete Python program implementing the RAG pipeline.

The code should run without errors on the selected test document.

Deliverable 2 — Sample Output

Provide at least **5 question-answer pairs**.

Each answer should show the source document/chunks used.

Example:

Question 1

Answer

Source

Question 2

Answer

Source

...

Deliverable 3 — Short Report

Prepare approximately a **2-page report** explaining:

- RAG architecture.
- Why FAISS was selected.
- Why the particular chunk size was selected.
- Why the embedding model was selected.
- How retrieval works.
- How the LLM generates the final answer.

These requirements are specified in the assignment's deliverables section.

Deliverable 4 — Experiment

Run the same three questions using:

k = 1

k = 3

$k = 5$

Compare:

- Retrieved information.
- Answer quality.
- Relevance.
- Completeness.

Prepare a comparison table.

73. Viva Questions and Answers

Q1. What is RAG?

Answer:

RAG stands for Retrieval-Augmented Generation. It retrieves relevant information from documents and provides that information to an LLM to generate an answer.

Q2. Why is RAG used?

Answer:

RAG allows an LLM to use external documents and helps generate answers based on current or private information.

Q3. What is an embedding?

Answer:

An embedding is a numerical vector representation of text that captures its semantic meaning.

Q4. What is FAISS?

Answer:

FAISS is a library used for efficient similarity search of vectors.

Q5. Why do we split documents?

Answer:

Large documents are divided into smaller chunks so that relevant pieces can be retrieved efficiently.

Q6. What is chunk size?

Answer:

Chunk size determines the approximate amount of text contained in each chunk.

Q7. What is chunk overlap?

Answer:

Chunk overlap is the amount of text repeated between consecutive chunks to preserve context.

Q8. What does k mean in retrieval?

Answer:

k represents the number of most relevant chunks retrieved for a query.

Q9. What does $k=3$ mean?

Answer:

It means the system retrieves the top three relevant chunks.

Q10. What is the role of the embedding model?

Answer:

It converts document text and user queries into numerical vectors so that semantic similarity can be calculated.

Q11. What is the role of the LLM?

Answer:

The LLM uses the retrieved context and user question to generate the final natural-language answer.

Q12. Does the LLM perform retrieval?

Answer:

No. The retriever performs document retrieval. The LLM generates the answer using the retrieved information.

Q13. What is hallucination?

Answer:

Hallucination occurs when an LLM generates information that is unsupported or incorrect.

Q14. How can hallucination be reduced in RAG?

Answer:

Use a context-restricted prompt and a low temperature, and instruct the LLM to say that it does not know when the information is not available.

Q15. Why is source information important?

Answer:

It helps us verify where the answer came from and improves transparency.

Q16. What happens if k is too small?

Answer:

Important information may not be retrieved.

Q17. What happens if k is too large?

Answer:

Too much irrelevant or repetitive information may be given to the LLM.

Q18. What is the difference between RAG and fine-tuning?

Answer:

RAG retrieves external information at query time, while fine-tuning modifies the model using training data.

Q19. What is cosine similarity?**Answer:**

Cosine similarity measures the similarity between two vectors based on the angle between them.

Q20. What is the main debugging step in RAG?**Answer:**

Check the retrieved chunks first. If the correct information was not retrieved, the LLM cannot reliably generate the correct answer from that information.

74. Result

After successful execution, the system should be able to:

- Load a document.
 - Split the document into chunks.
 - Generate embeddings.
 - Store embeddings in FAISS.
 - Retrieve relevant chunks for a user query.
 - Send retrieved context to an LLM.
 - Generate a grounded answer.
 - Display the source documents used.
-

75. Conclusion

In this experiment, we implemented a basic **Retrieval-Augmented Generation system** using document loading, text chunking, embeddings, FAISS vector storage, similarity-based retrieval and an LLM.

The experiment demonstrates that an LLM does not always need to rely only on its internal knowledge. It can retrieve relevant information from an external document and use that information to generate a better, more grounded response.

The most important concept to remember is:

RAG = Retrieve Relevant Information

+

Generate an Answer

The quality of a RAG system depends strongly on:

Document Quality

↓

Chunking

↓

Embeddings

↓

Retrieval

↓
Prompt

↓
LLM

As highlighted in the assignment, **poor chunking can lead to poor retrieval, and poor retrieval can lead to poor answers**. Therefore, students should experiment with chunk size, overlap and k rather than assuming that one configuration will work for every document.
